ProBLAST – Basic Local Alignment Tool for Probabilistic DNA Sequences

Yazdan Zinati – 260928077

December 11, 2024

Introduction

The power to compare and match genomic sequences by efficiently aligning them is a central part of many studies in computational biology. As a result, researchers have developed many alignment tools throughout the years, each with their own specific application, advantages, and disadvantages. Arguably, the most well known of such tools in BLAST (Basic Local Alignment Search Tool) [1]. By aligning a short query sequence to a long genome database, BLAST allows researchers to examine evolutionary patterns and identify homologous regions, which often is the first step in inferring functional relationships. However, vanilla BLAST and many other sequence alignment approaches function under the assumption that the reference genome database is deterministic and never ambiguous at certain positions. That assumption does not hold when non-deterministic scenarios arise because of errors, polymorphism, etc. One intriguing such scenario is when the reference database is constructed from the output of an ancestral sequence reconstruction algorithm where each position of the sequence is assigned a probability of being one of the four nucleotides. Developing a ProBabilistic Local Alignment Search Tool (our method called ProBLAST) requires rethinking the principles of sequence alignment when it comes to defining what is an optimal alignment of a non-probabilistic query to a probabilistic sequence database. With the increase in complexity of the sequence database, it also involves performing such alignment efficiently. Since BLAST is relatively simple, robust, traceable and shown to be highly adaptable to a variety of contexts (as shown by the existence of variations such as tblastn, tblastx, blastp, blastx, etc.) [2], we decided to solve the introduced problem by developing a probabilistic generalization of BLAST. In this paper, we explore alignment scoring and optimization criteria for probabilistic alignment, implement proposed algorithm, and evaluate performance and time complexity on a reference sequence database constructed from a portion of chromosome 22 of the predicted BoreoEutherian ancestor sequence.

Methodology

Inputs

ProBLAST is designed to handle scenarios where the reference sequencing data is associated with varying levels of confidence, while the query remains deterministic. The input to the algorithm comprises two key elements: 1. Query sequence and 2. Probabilistic database. The query sequence is a DNA string of ACGT nucleotides. We discuss our synthetic query generation procedure in the Results section.

The algorithm takes in the probabilistic database as two files:

1. Sequence file: containing the reference sequence of size L as a nucleotide string.

2. Confidence file: certainty of the associated nucleotide in the sequence file being correct. Note that the nucleotide appearing in position i of the sequence has the highest probability (P_i^{max}) of being correct. The remaining probability $(1-P_i^{max})$ is equally distributed among the three nucleotides that do not appear in the sequence at position i as $(1-P_i^{max})/3$. However, the input function can trivially be modified to only accept the confidence of each nucleotide as a matrix of size $4\times L$. Below is an example of such matrix for conceptualization where providing the sequence wouldn't be necessary.

Table 1: Example of input confidence matrix

	1	2	3	4
A	0.9	0.8	0.05	0.3
C	0.1	0.0	0.025	0.40
G	0.0	0.1	0.025	0.18
T	0.0	0.1	0.9	0.12

Indexing

The first step of deploying or extending BLAST on a new reference sequence is indexing. Like BLAST, ProBLAST operates under the assumption that an optimal local alignment will statistically contain w consecutive high probability nucleotides in the reference.

Thus, we index our reference similar to BLAST by using the sliding window approach to extract w-mers from the probabilistic sequence. For each w-mer, the most likely nucleotide $P[i][j]$ is selected where i is the index corresponding to one of the four nucleotides and j is the position along the sequence (See Table 1).

$$wmer[k]=argmax_{i\in A,C,G,T}P[i,j+k],k=0,1,2,\ldots,w-1$$

In addition to storing the start position j of w-mers, we calculate and store a confidence score associated with the w-mer computed as:

$$Confidence_j = C_d(j) = \prod_{k=0}^{w-1} P(argmax_{i \in A,C,G,T} P[i, j+k, j+k])$$

This confidence score reflects the likelihood of the w-mer being correct. We introduced a confidence threshold hyperparameter T that could be used to only include w-mers consisting of reliable parts of the sequence with $Confidence_j > T$. Chosing a strict threshold can be beneficial to ensure building a robust index with very long sequences. In our application, we chose a moderate threshold of 0.6.

Although we considered including more than a single high-confidence w-mer for each window, this would have considerably increased the size of the index and the time to construct it. However, we can revisit this approach in the future given that we have defined a confidence score.

Seeding

The seeding stage involves identifying exact or close matches between the query and the database sequences using the indexed w-mers. Here, we adapted this step by scoring each candidate probabilistically to account for uncertainties in the reference sequence. We extract w-mers by sliding a window of length w along the query sequence. The algorithm then retrieves the w-mers' starting positions and confidence scores (discussed during index construction) if it exists in the index. To score the quality of found matches, we compute a probabilistic match score by using the confidence of the portion of the reference that matches with a query w-mer. To score candidates, we define an alignment score which we will continue to use throughout this paper:

$$Score(q, d, j) = \sum_{k=0}^{w-1} P[i, j+k], if\ q[k] = argmax_{i \in A,C,G,T} P[i, j+k]$$

$$Score(q, d, j) = \sum_{k=0}^{w-1} -P[i, j+k], if\ q[k] \neq argmax_{i \in A,C,G,T} P[i, j+k]$$

,where q is a query w-mer, d is a reference database w-mer at starting position j. $P[i, j+k]$ is the probability of nucleotide i at position $j+k$ in the reference being correct. Therefore, the introduced scoring system can be interpreted as the likelihood of the w-mer aligning with the reference segment. This likelihood rewards and penalizes matches and mistatches proportionally to the confidence value.

The computed score for each candidate w-mer is then scaled by the confidence score of the database w-mer $S_{weighted}(q, d, j) = Score(q, d, j) \times C_d(j)$. We only consider candidates to be strong if their weighted score exceeds a certain scoring threshold ensuring that we identify

candidates even in noisy or low-confidence portions of the database. Once again, we chose a moderate threshold of 0.5 since this specific database isn't very long. However, a stricter threshold can help reduce possible low-confidence candidates that will have to be processed at later stages.

We also implemented and explored an alternate but more exhaustive seeding strategy, which we later found quite inefficient and therefore not ultimately used. This involved iterating over all the indexed w-mers and comparing them all against the sliding query w-mer using the defined scoring mechanism. This approach has the capability of considering partial matches between query and database w-mers. Such approach could be viable if we increase the parameter w (which in turn, decreases the entries in the index). Increasing w in this case wouldn't result in low sensitivity like vanilla BLAST since partial matches are allowed. But tuning for an optimal w would be time-consuming.

Ungapped Extension

ProBLAST's ungapped extension phase in similar to BLAST but uses a different alignment scoring. In this step, we refine found candidates in the previous step and extend them in both left and right direction to find high-scoring segment pairs (HSP). The extension proceeds until the match breaks or we reach a predefined threshold for the score (maximum reached extension score). During extension, the score is increased by the match probability if the query q_i and reference nucleotides r_i at position i match and decreased by the mismatch probability if they mismatch:

$$Match\ Score = P(r_i = q_i) \qquad Mismatch\ Score = -(1 - P(r_i = q_i))$$

In simpler terms, the matching and mismatch reward/penalties are proportional to the likeliness of reference nucleotide and query nucleotide at position i matching.

Gapped Extension

In this step, we refine high-scoring ungapped alignments by allowing gaps. In our approach, we use the top 10 best-scoring ungapped extension candidates. Gapped extension is performed using a variation of Needleman–Wunsch dynamic programming algorithm in each direction. Filling in the dynamic programming matrix and trace back is not much different from regular blast with the distinction of new probabilistic match and mismatch reward/penalty the calculation of which we explained in the ungapped extension phase section.

$$Score(i,j) = max \begin{matrix} Score(i-1,j-1) + Match/Mismatch\ Score(i,j) \\ Score(i-1,j) - gap\ penalty \\ Score(i,j-1) - gap\ penalty \end{matrix}$$

We chose a gap penalty of 0.5. The second and third lines of the equality correspond to deletion in the reference and deletion in the query, respectively.

Results

Query Construction

Since we do not have an experimental query sequence to evaluate ProBLAST (on chromosome 22 of BoreoEutherian ancestor sequence), we simulate a query sequence following a probabilistic model derived from the database. We essentially want to simulate a descendant sequence of the reference that has gone through random mutations, deletions, insertions, etc. This is done by randomly picking a starting position within the reference sequence (from 0 to L_r - L_q). Then, a sequence of predefined length is constructed by sampling nucleotides at each position reflecting the probability distribution of the reference sequence ensuring that we capture biological heterogeneity in the query. To simulate mutations in the query, we perform substitution, insertion, and deletions following user-defined substitution, insertion, and deletion rates in percentage. An improvement in query construction is possible by also controlling for the length of subsequent deletions and insertions and performing substitutions following a substitution matrix that assigns different probabilities to transitions and transversions.

Sanity Checking

To sanity check ProBLAST, we set the substitution, deletion, and insertion rates to zero and instead of sampling nucleotides following their probability distribution in the reference, we sampled them as the nucleotide with the highest confidence. This ensured that the query is identical to a portion of the database. As expected, the highest scoring alignment was a perfect one (without any gaps) starting from the true randomly-picked reference starting position across all 3 runs. Here, we selected the query length to be 100. The output of these runs can be found in `sanity_check.txt`.

Evaluation

An intuitive measure of correctness of ProBLAST could come from examining whether the true randomly-picked start position during query construction can be found within the bounds of found alignment. Although, this measure is imperfect as the deletion and insertion rates can change the length of the query, we select equal insertion and deletion rates. We could potentially correct for false positives, but we do not expect a large number them due to the size of the reference sequence (604466) compared to our queries of fixed length 500.

We select 4 configurations for query constructions with different substitution, deletion, and insertion rates. All parameters used for this study are listed in Appendix Table 1. We run each experiment 5 times are report how many times the correct alignment was found (following previously described criteria) and report the mean and standard deviation score of the following metrics in Table 2 (see out_s1.txt, ..., out_s4.txt for the raw program output of each experiment):

N-gram similarity score: Jaccard index between the two sequences corresponding to n-gram overlap similarity between reference and query

LCS similarity score: longest subsequence of characters common to both the query and reference normalized by query length. A high LCS score is desired.

Sequence Entropy: Scores the uncertainty in the aligned reference sequence at positions where the query sequence has a nucleotide.

Runtime: running time of ProBLAST (excluding indexing).

We chose the above metrics to comparatively score the goodness of our alignments between the 4 setups without using the score used to infer the alignments themselves. Overall, we observe a decrease in correctness and other alignment scoring metrics (n-gram, LCS, entropy) when we introduce more mutations in the query. The runtime does not seem to follow any meaningful pattern.

Experimenting with parameters

With substitution, deletion, and insertion rates set to 5%, we experiment with query sizes 100, 500, and 1000 (see correctness results across 10 runs reported in Table 3). We observe the performance getting better with longer queries and the runtime increasing (seemingly polynomially in accordance with the O(mn) time complexity of Needleman-Wunsch where m and n are the lengths of the two input sequences). This makes sense as the algorithm has more information to find a better alignment.

Table 4 shows the runtime and performance (in terms of correctness) of ProBLAST with varying w parameters. As expected, we achieve increased sensitivity at the cost of increased runtime with a smaller w.

Table 2: Result of running ProBLAST under different query generation setups and reporting 5 metrics

	N-gram similarity	LCS similarity	Sequence Entropy	Correctness (out of 5)	Runtime (seconds)
Substitution rate = 2% Deletion rate = 2% Insertion rate = 2%	0.395 +/- 0.152	0.797 +/- 0.127	3.024+/- 0.460	4	18.919 +/- 1.874
Substitution rate = 5% Deletion rate = 5% Insertion rate = 5%	0.242 +/- 0.053	0.660+/- 0.101	3.435+/- 0.347	4	19.141 +/- 1.174
Substitution rate = 10% Deletion rate = 10% Insertion rate = 10%	0.174 +/- 0.040	0.612 +/- 0.077	3.641+/- 0.199	3	15.097+/- 3.593
Substitution rate = 15% Deletion rate = 15% Insertion rate = 15%	0.143 +/- 0.011	0.562+/- 0.012	3.803+/- 0.058	2	10.257 +/- 3.868

Table 3: Result of running ProBLAST with varying query sizes

Query sizes	Correctness (out of 10)	Average runtime (seconds)
100	5	1.2
500	8	18.131
1000	10	71

Table 4: Result of running ProBLAST with varying w sizes

w	Correctness (out of 10)	Average Total runtime (including index building
6	8	70.60
12	8	30.44
20	7	25.45

Discussion

We presented a strategy of extending BLAST to work on probabilistic reference databases and showed good performance attempting to align a synthetic query that has gone through mutations. ProBLAST and many other methods are a testament to the highly adaptable framework of BLAST. The major disadvantage of ProBLAST is the sheer number of tunable parameters (index threshold, score threshold, confidence threshold, etc.), most of which we selected to be arbitrary thresholds. Tuning these parameters can improve the performance of the method. However, it has to be done all over when applying the method to a new dataset, which can be time-consuming. Like BLAST, ProBLAST is linear in the size of the reference database or O(n). We suggested possible improvements in the indexing and seeding stages of ProBLAST which without optimization, come at great cost to time complexity. Such optimizations can be a good future direction. We can also study probabilistic sequence alignment tools to potentially replace our probabilistic version of the Needleman-Wunsch algorithm in the extension stages. Finally, we need to perform a literature review of existing probabilistic local alignment methods to get familiarized with common practices in this field and compare our methods in terms of performance.

Appendix

Table A1: Parameters used for evaluation

Parameter	Value
w	12
Threshold (for w-mer index creation)	0.90
Query length	500
Score Threshold	0.5
Confidence Threshold	0.6
Gap Penalty	0.5

Bibliography

[1] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," Journal of Molecular Biology, vol. 215, no. 3, pp. 403–410, Oct. 1990, doi: https://doi.org/10.1016/S0022-2836(05)80360-2.

[2] D. W. Mount, "Using the Basic Local Alignment Search Tool (BLAST)," *Cold Spring Harbor Protocols*, vol. 2007, no. 7, p. pdb.top17, Jul. 2007, doi: https://doi.org/10.1101/pdb.top17.